

Assignment ::- 11

Name	Abhineet Tiwari
Roll No:	20965
Subject	DBMS

Q1) Create a MySQL database named SalesDB and a table named OrderData.

OrderData Table

Column Name	Data Type	Constraint
OrderID	INT	PRIMARY KEY
CustomerName	VARCHAR(50)	NOT NULL
Email	VARCHAR(80)	NOT NULL
City	VARCHAR(30)	NOT NULL
ProductCategory	VARCHAR(30)	NOT NULL
OrderDate	DATE	NOT NULL
OrderAmount	DECIMAL(10,2)	NOT NULL
OrderStatus	VARCHAR(20)	NOT NULL

Insert at least 12 records into the table.

-- Step 1: Create the database

```
CREATE DATABASE IF NOT EXISTS SalesDB;
```

-- Step 2: Select the database

```
USE SalesDB;
```

-- Step 3: Create the OrderData table

```
CREATE TABLE OrderData (
```

```
    OrderID INT PRIMARY KEY,
```

```
    CustomerName VARCHAR(50) NOT NULL,
```

```
    Email VARCHAR(80) NOT NULL,
```

```
    City VARCHAR(30) NOT NULL,
```

```
    ProductCategory VARCHAR(30) NOT NULL,
```

```
    OrderDate DATE NOT NULL,
```

```
OrderAmount DECIMAL(10,2) NOT NULL,

OrderStatus VARCHAR(20) NOT NULL

);

-- Step 4: Insert 12 records

INSERT INTO OrderData

(OrderID, CustomerName, Email, City, ProductCategory,

OrderDate, OrderAmount, OrderStatus)

VALUES

(1, 'Rahul Sharma', 'rahul@gmail.com', 'Mumbai',

'Electronics', '2026-01-05', 55000.00, 'Delivered'),

(2, 'Priya Patil', 'priya@gmail.com', 'Pune',

'Furniture', '2026-01-08', 32000.00, 'Delivered'),

(3, 'Aman Verma', 'aman@gmail.com', 'Nashik',

'Clothing', '2026-01-12', 8500.00, 'Pending'),

(4, 'Sneha Joshi', 'sneha@gmail.com', 'Mumbai',

'Electronics', '2026-01-20', 72000.00, 'Delivered'),

(5, 'Karan Mehta', 'karan@gmail.com', 'Thane',

'Furniture', '2026-02-03', 28000.00, 'Cancelled'),

(6, 'Neha Shah', 'neha@gmail.com', 'Pune',

'Electronics', '2026-02-08', 48000.00, 'Delivered'),

(7, 'Rohan Desai', 'rohan@gmail.com', 'Mumbai',

'Clothing', '2026-02-15', 12000.00, 'Pending'),

(8, 'Meera Kulkarni', 'meera@gmail.com', 'Nashik',

'Furniture', '2026-02-22', 36000.00, 'Delivered'),

(9, 'Arjun Nair', 'arjun@gmail.com', 'Thane',

'Electronics', '2026-03-02', 64000.00, 'Shipped'),

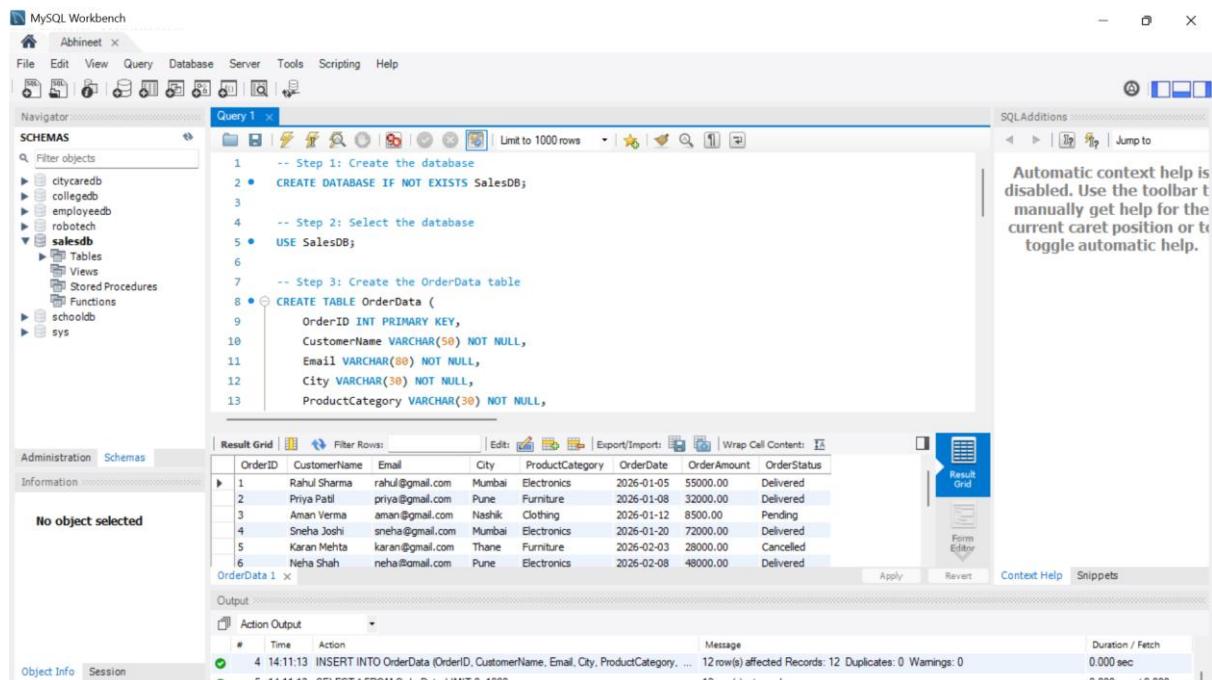
(10, 'Divya More', 'divya@gmail.com', 'Mumbai',

'Furniture', '2026-03-07', 41000.00, 'Delivered'),
```

```
(11, 'Isha Singh', 'isha@gmail.com', 'Pune',
'Clothing', '2026-03-12', 15000.00, 'Shipped'),
(12, 'Om Jadhav', 'om@gmail.com', 'Mumbai',
'Electronics', '2026-03-18', 59000.00, 'Pending');
```

-- Step 5: Display all records

```
SELECT * FROM OrderData;
```



Q2) Create the following MySQL indexes on the OrderData table:

Index Name	Column	Index Type
idx_city	City	Normal index
idx_email	Email	Unique index
idx_order_date	OrderDate	Normal index

Display all indexes using SHOW INDEX.

```
USE SalesDB;
```

-- 1. Create a normal index on City

```
CREATE INDEX idx_city
```

```
ON OrderData(City);
```

-- 2. Create a unique index on Email

```
CREATE UNIQUE INDEX idx_email
```

```
ON OrderData(Email);
```

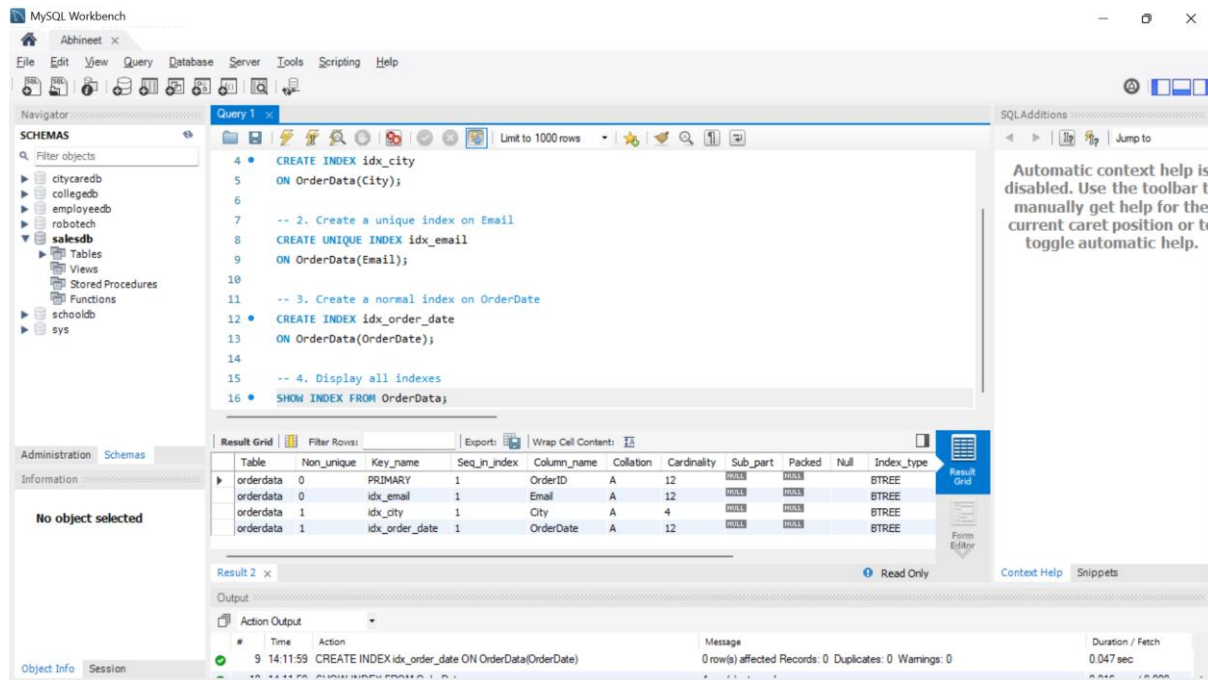
-- 3. Create a normal index on OrderDate

CREATE INDEX idx_order_date

ON OrderData(OrderDate);

-- 4. Display all indexes

SHOW INDEX FROM OrderData;



Q3) Create a composite index named idx_city_order_date.

The index should contain:

Index Position	Column
1	City
2	OrderDate

Write a query that searches for Mumbai orders within a specified date range

USE SalesDB;

-- Step 1: Create the composite index

CREATE INDEX idx_city_order_date

ON OrderData(City, OrderDate);

-- Step 2: Search for Mumbai orders within a date range

SELECT

OrderID,

CustomerName,

City,

OrderDate,

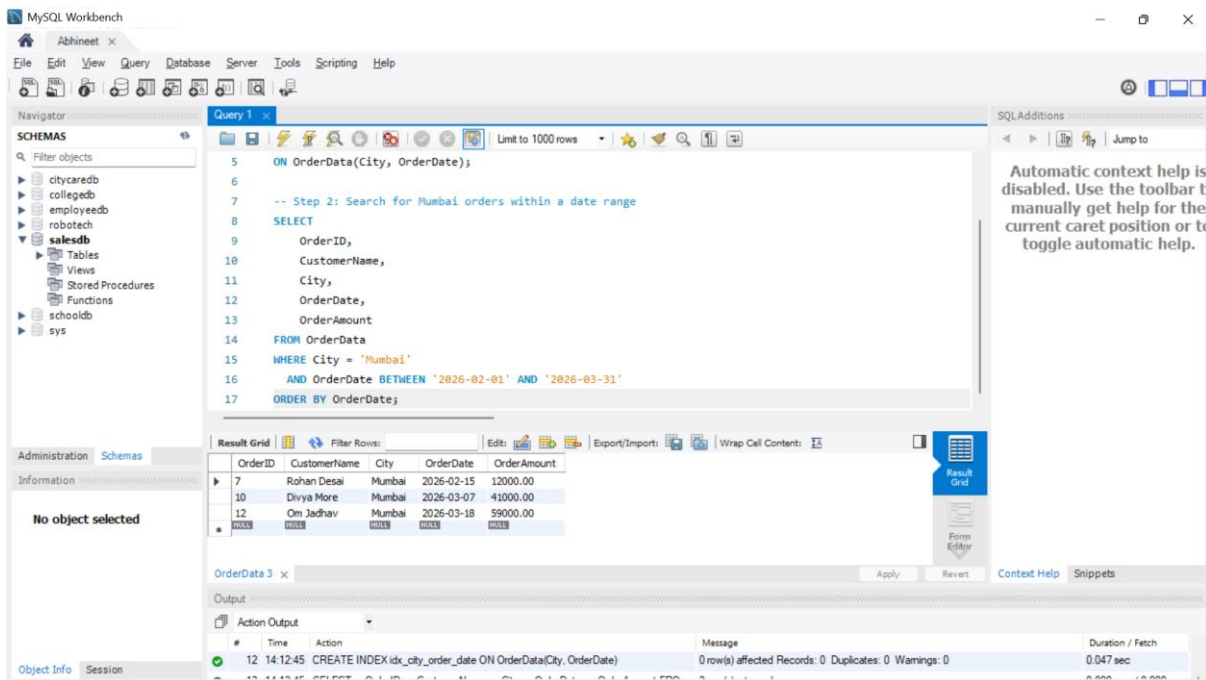
OrderAmount

FROM OrderData

WHERE City = 'Mumbai'

AND OrderDate BETWEEN '2026-02-01' AND '2026-03-31'

ORDER BY OrderDate;



Q4) Write a MySQL EXPLAIN query to analyze the following search:

SELECT *

FROM OrderData

WHERE City = 'Mumbai';

Record and explain the following output columns:

EXPLAIN Column
type
possible_keys
key
rows
Extra

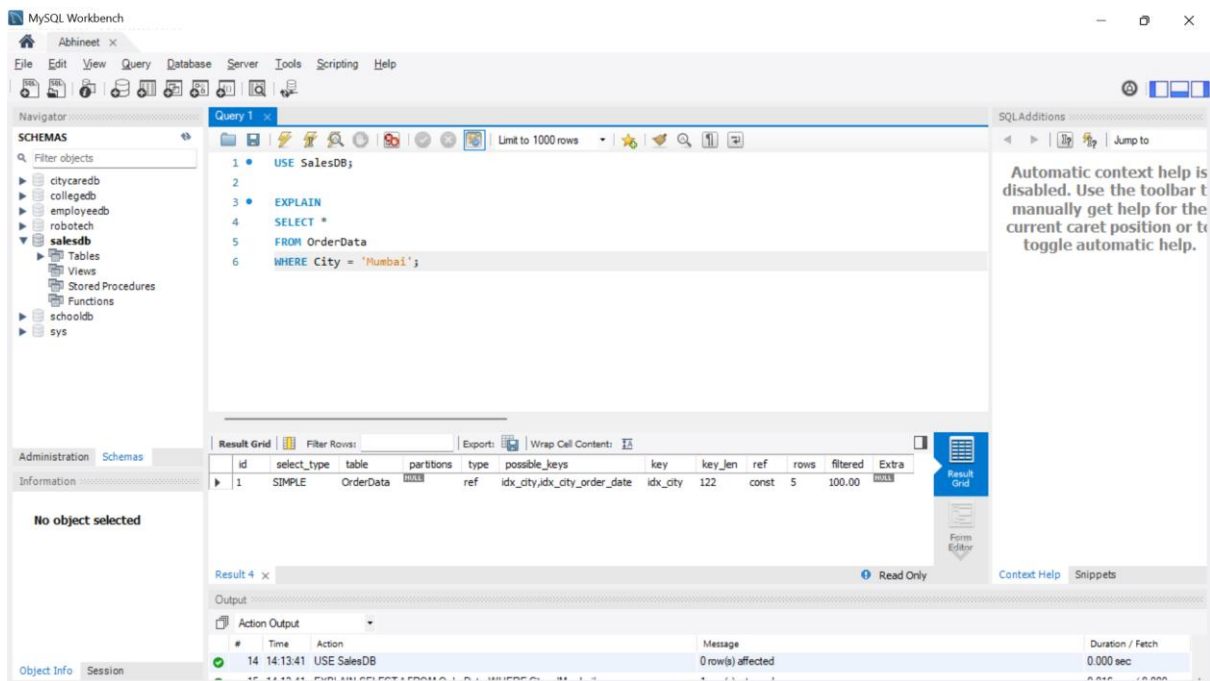
USE SalesDB;

EXPLAIN

SELECT *

FROM OrderData

WHERE City = 'Mumbai';



Q5) Compare query performance before and after creating an index on City.

Perform the following steps:

Step	Task
1	Drop idx_city, if it exists
2	Run EXPLAIN on the City search
3	Create idx_city
4	Run the same EXPLAIN query again
5	Compare the selected index and estimated rows

USE SalesDB;

-- Step 1: Drop idx_city

DROP INDEX idx_city ON OrderData;

-- Step 2: EXPLAIN before index

EXPLAIN

SELECT *

FROM OrderData

WHERE City = 'Mumbai';

-- Step 3: Create idx_city

CREATE INDEX idx_city

ON OrderData(City);

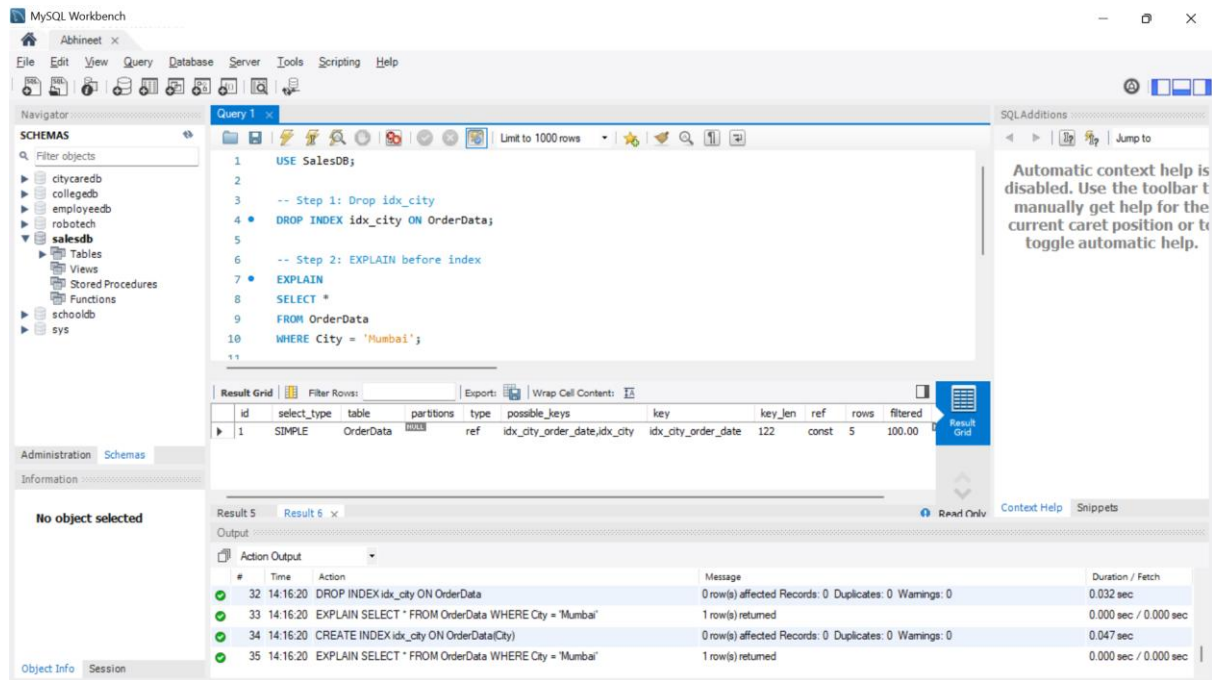
-- Step 4: EXPLAIN after index

EXPLAIN

SELECT *

FROM OrderData

WHERE City = 'Mumbai';



Q6) Rewrite the following query in a more index-friendly form:

SELECT *

FROM OrderData

WHERE YEAR(OrderDate) = 2026;

The improved query should:

Requirement
Avoid applying a function to OrderDate
Use a start date
Use an end date
Select only OrderID, CustomerName, OrderDate, and OrderAmount
Be analyzed using EXPLAIN

USE SalesDB;

EXPLAIN

SELECT

OrderID,

CustomerName,

OrderDate,

OrderAmount

FROM OrderData

WHERE OrderDate >= '2026-01-01'

AND OrderDate < '2027-01-01';

The screenshot shows the MySQL Workbench interface. The 'Query' window contains the following SQL code:

```

1  USE SalesDB;
2
3  EXPLAIN
4  SELECT
5      OrderID,
6      CustomerName,
7      OrderDate,
8      OrderAmount
9  FROM OrderData
10 WHERE OrderDate >= '2026-01-01'
11       AND OrderDate < '2027-01-01';

```

The 'Result Grid' shows the execution plan for the query:

#	select_type	table	partitions	type	possible_keys	key	key_len	ref	rows	filtered	Extra
1	SIMPLE	OrderData		ALL	idx_order_date				12	100.00	Using where

The 'Output' window shows the execution log:

#	Time	Action	Message	Duration / Fetch
34	14:16:20	CREATE INDEX idx_city ON OrderData(City)	0 row(s) affected Records: 0 Duplicates: 0 Warnings: 0	0.047 sec
35	14:16:20	EXPLAIN SELECT * FROM OrderData WHERE City = 'Mumbai'	1 row(s) returned	0.000 sec / 0.000 sec
36	14:18:32	USE SalesDB	0 row(s) affected	0.000 sec
37	14:18:32	EXPLAIN SELECT OrderID, CustomerName, OrderDate, OrderAmount F...	1 row(s) returned	0.000 sec / 0.000 sec